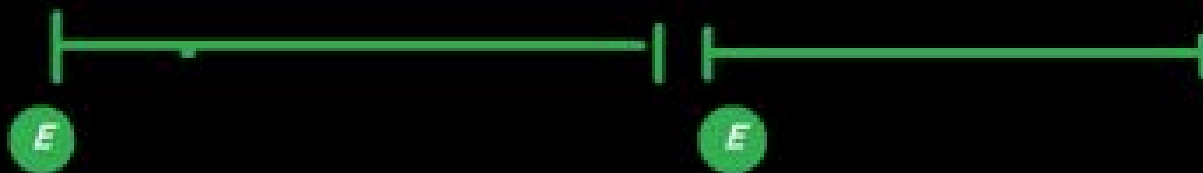
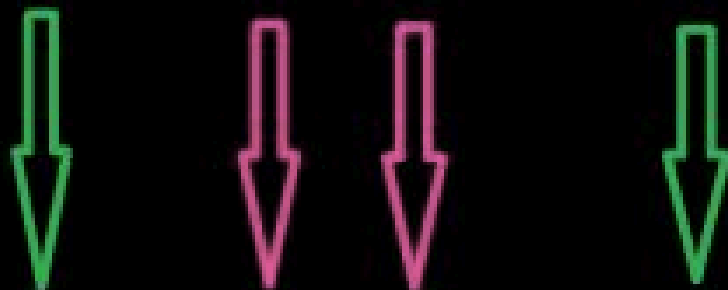


Debouncing and Throttling in javascript

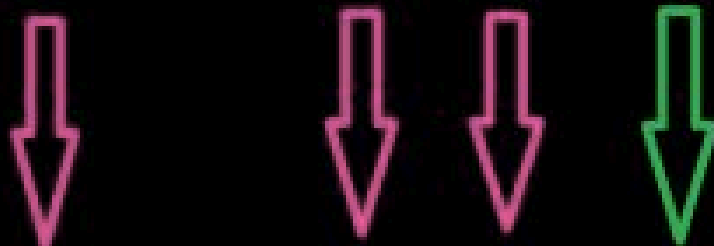
THROTTLE

click *click* *click* *click*



DEBOUNCE

click *click* *click* *click*

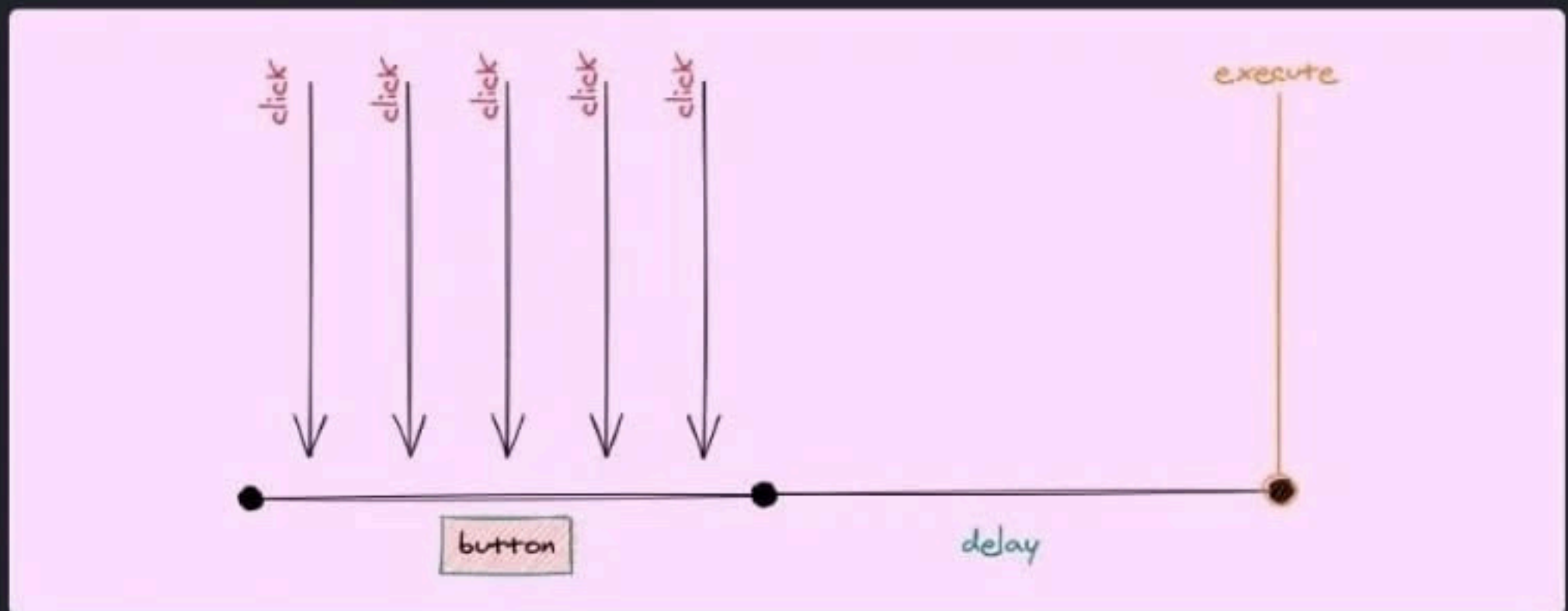


What is debounce?

button



If we click on the button many times, there are going to be chances of inresponsive(freezed) UI and there will be too many request API calls.



Debouncing is a programming practice that we use to ensure that certain tasks don't fire too often.

How does debouncing work ?

if we click on the button many times, we will not execute the action that the button is firing until we stop clicking on it, and the delay we set has passed. When delay passes, we execute the action. Here is how we can write that in Javascript

A Debounce function is a higher-order function that returns another function, to create closure around the function parameters (func, timeout) and the timer variable.

 [index.html](#)

```
<button id="btn">SUBMIT</button>
```


● ● ● debounce.js

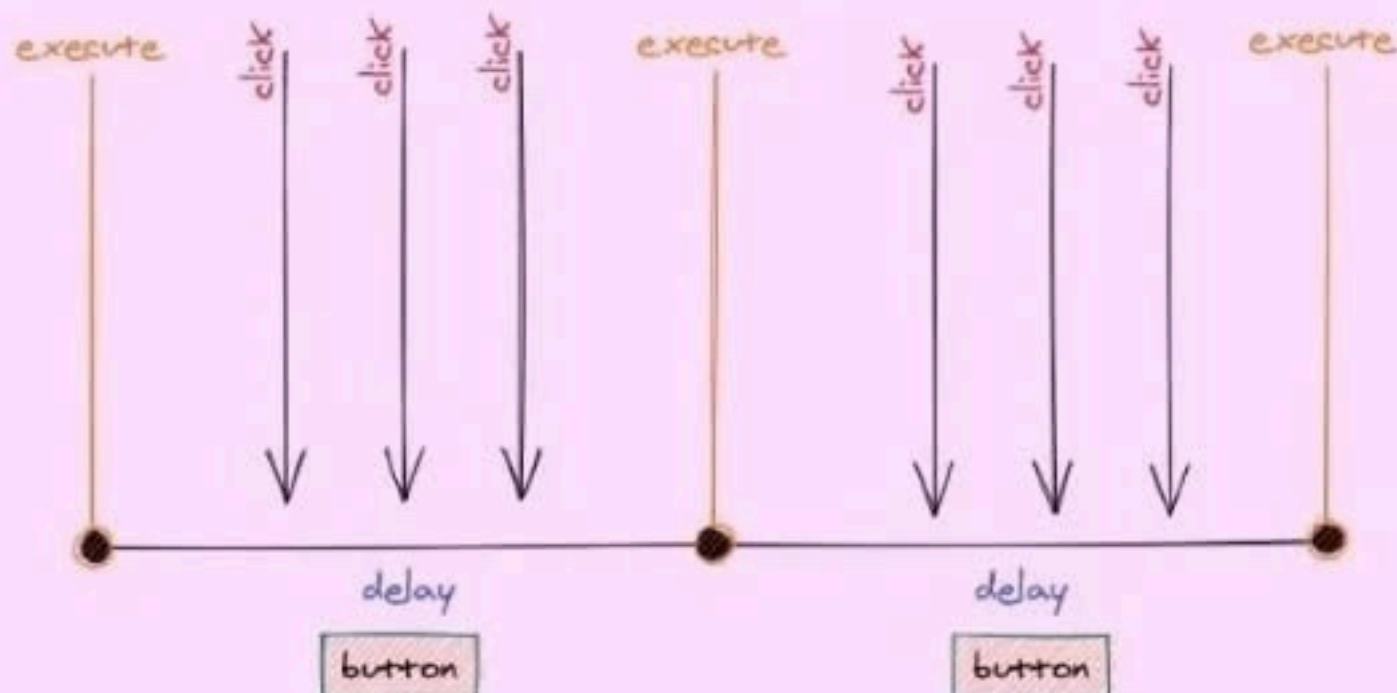
```
const debounce = (fn, delay) => {  
  let timeoutID;  
  
  return function (...args) {  
    if (timeoutID) {  
      clearTimeout(timeoutID);  
    }  
  
    timeoutID = setTimeout(() => {  
      fn(...args);  
    }, delay);  
  };  
};  
  
document.getElementById('btn').addEventListener(  
  'click',  
  debounce((e) => {  
    console.count('click');  
  }, 1000)  
);
```

What is throttle ?

button



Unlike debounce, the throttle is firing event between intervals, so when we click on a button for the first time it will execute the action, and if we keep clicking on that button, it will not fire event until time interval has passed, and if you keep clicking it will fire every time interval it completes.



Throttling example

● ● ● throttle.js

```
const throttle = (fn, delay) => {  
  let last = 0;  
  return (...args) => {  
    const now = new Date();  
    if (now - last < delay) {  
      return;  
    }  
    last = now;  
    return fn(...args);  
  };  
};  
  
document.getElementById('btn').addEventListener(  
  'click',  
  throttle((e) => {  
    console.count('click');  
  }, 2000)  
);
```